

Relatório: Aplicação do Docker no Ambiente de Desenvolvimento

Projeto: Pizzaria Callidus **Escopo:** Ambiente de desenvolvimento local (Caso 1)

1. Objetivo

Padronizar o ambiente de desenvolvimento do projeto, eliminando inconsistências de versão do Node.js entre as máquinas dos colaboradores e o ambiente usado no CI/CD (GitHub Actions), que já roda em Node 20.

2. Contexto identificado antes da mudança

- Projeto React 19 + Vite 8 + TypeScript, com PWA (Workbox).
- Ambiente local usava a versão do Node instalada em cada máquina (ex.: Node 22 na máquina de desenvolvimento testada), enquanto o workflow de deploy (.github/workflows/deploy.yml) fixa Node 20.
- Não havia mecanismo que garantisse que todos os colaboradores rodassem a mesma versão de Node/npm.

3. O que foi implementado

Três arquivos novos foram criados na raiz do projeto:

Arquivo	Função
Dockerfile.dev	Define a imagem do ambiente de desenvolvimento: Node 20, instalação de dependências (npm ci) e execução do vite --host
docker-compose.dev.yml	Orquestra o container, mapeia a porta 5173 e monta o volume para hot-reload
.dockerignore	Evita copiar node_modules, .git, dist, docs e outros arquivos desnecessários para dentro da imagem

4. Comparativo: fluxo antes x depois

Etapas	Antes (sem Docker)	Depois (com Docker)
Pré-requisito na máquina	Ter Node.js instalado (versão variável por pessoa)	Ter o Docker Desktop instalado e em execução
Instalar dependências	npm i	Não é feito manualmente — ocorre automaticamente dentro do build da imagem (npm ci no

Rodar o projeto	<code>npm run dev</code>	Dockerfile.dev) <code>docker compose -f docker-compose.dev.yml up --build</code>
Versão do Node usada	A que estiver instalada localmente (podia divergir do CI)	Sempre Node 20, igual ao CI, garantida pela imagem
Atualizar dependências após mudança no package.json	<code>npm i</code> de novo (automático, sempre pega o mais recente)	Necessário lembrar de rodar com a flag <code>--build</code> , senão o Docker reutiliza a imagem em cache com dependências antigas
Rodar comandos avulsos (lint, testes)	<code>npm run lint</code> direto	Pode continuar rodando localmente, ou dentro do container via <code>docker compose exec frontend npm run lint</code>
Acesso à aplicação no navegador	<code>http://localhost:5173/Pizzaria-Callidus/</code>	O mesmo endereço — a linha “Network” exibida pelo Vite deve ser ignorada, pois refere-se ao IP interno do container, não acessível fora da rede do Docker

5. Passo a passo para rodar o ambiente

```
git clone <repositório>
cd Pizzaria-Callidus
docker compose -f docker-compose.dev.yml up --build
```

Acessar no navegador:

`http://localhost:5173/Pizzaria-Callidus/`

(o caminho `/Pizzaria-Callidus/` é obrigatório, definido pela propriedade `base` no `vite.config.ts`, pensada originalmente para o deploy no GitHub Pages)

6. Problemas encontrados durante a implementação e soluções

Problema	Causa	Solução
empty compose file	Arquivo <code>docker-compose.dev.yml</code> salvo sem conteúdo válido (possível problema de encoding no	Reescrever/salvar o arquivo garantindo conteúdo e encoding UTF-8

	Windows)	
failed to connect to the docker API	Docker Desktop não estava em execução	Abrir o Docker Desktop e aguardar sua inicialização completa antes de rodar o comando
URL "Network" não abre no navegador	Confusão entre o IP interno do container (rede virtual do Docker) e o endereço acessível pela máquina host	Usar sempre o endereço "Local" (localhost:5173), ignorando a linha "Network" ao rodar via Docker

7. Conclusão

O ambiente de desenvolvimento passou a ser reprodutível independentemente da máquina do colaborador. A principal troca de responsabilidade foi: deixou de ser necessário gerenciar a versão do Node.js localmente, mas passou a ser necessário ter o Docker instalado e lembrar de usar a flag `--build` sempre que as dependências do projeto (`package.json`) forem alteradas.

8. Próximos passos possíveis

- Aplicar a mesma lógica de padronização de ambiente dentro do workflow de CI/CD (`deploy.yml`), reaproveitando a mesma definição de imagem usada em desenvolvimento.
- Caso o projeto migre de hospedagem (do GitHub Pages para um servidor próprio), avaliar um `Dockerfile` de produção com build multi-stage finalizando em um servidor Nginx.